

Metaphors for Electronic Music Production in *Reason* and *Live*

Matthew Duignan, James Noble, Pippin Barr[†] and Robert Biddle[‡]

[†] School of Mathematics and Computing Sciences [‡] Human Oriented Technology Laboratory
Victoria University of Wellington, New Zealand Carleton University, Ottawa, Canada
Matthew.Duignan@mcs.vuw.ac.nz Robert.Biddle@carleton.ca

Abstract. Electronic music production was originally accomplished using a variety of electronic components and conventional analogue recording techniques. Both the electronic components and the recording equipment are now being replaced by computer software. In this paper we present a comparative study of two popular new systems, *Reason* and *Live*, concentrating on the role of user-interface metaphors. We compare the two systems, identify the key ways metaphor is used, and describe how it affects usability of the systems focusing on the role that user-interface metaphors play in their design.

1 Introduction

Electronic music production was originally accomplished using a variety of electronic components and conventional analogue recording techniques. Now both the electronic components and the recording equipment are being replaced by computer software. We are interested in the user interface design for such software, and in this paper we present a comparative study of two popular new systems, *Reason* and *Live*, concentrating on the role of user-interface metaphors.

Most electronic music systems have similar functional structure, regardless of whether they are hardware or software. As sources of audio signals, there are *sound generators*, which can include synthesisers, recorded sound “loop” players, or drum machines. There are also *effects processors*, which filter, distort, compress, or otherwise modify an audio signal. To combine audio signals, *mixers* are used, that allow selection of sources, and setting of audio signal levels. Finally, there are *sequencers*, that control audio source behaviour over time.

We are interested in the design of electronic music software in particular both because the domain is highly specific and complex, and because elements of electronic music systems are essentially specialised programming systems. Our previous work in this area has focused on corpus analysis of one very specialised system for sound synthesis [1–3]. Much existing computer music software utilises interface metaphor to a striking degree. Recently, we have been working towards a principled model to guide understanding interface metaphor. This paper is a case study which utilises a simplified version of our framework. This serves the dual purpose of illuminating some of the interesting ways in which interface metaphor is being used in commercial software, and also acts as a proof of concept for our analytical model.

2 Metaphor

The concept of a user-interface metaphor is one which is widely known in the field of human-computer interaction and often utilised in the process of user-interface design. There has been advocacy and acceptance [4–7], and also expressions of caution [8, 9], and many other points of view. Many metaphors, such as the desktop metaphor and the trash-can metaphor, have become ubiquitous. In recent work, we have developed a principled model for understanding and evaluating user interface metaphor. In particular, we have explored the philosophical and semiotic foundations of user interface metaphor [10], and suggested heuristics based on the principles that can provide guidance for development and evaluation of user interface metaphors [11, 12].

In this paper we will explore the user interfaces of the two music systems using our heuristics to identify and evaluate their key metaphors. For each of the two systems, we first provide a general overview. We then identify several of the key metaphors involved in the interface. For each, we name the metaphor in small capitals (e.g. THE DATA IS A DOCUMENT) following the convention of Lakoff and Johnson [13]. We then identify the *representamen* of the metaphor; this involves the graphical and language elements that suggest the presence of the metaphor, and is a key idea from the semiotics of Pierce upon which our principled model is based [10]. Lastly we evaluate the metaphor, considering the issues that affect usability. We focus primarily on *structural* metaphors, where the structure of a real-world object is used to frame process elements in the user interface. The subset of the heuristics from our model that we consider follow. During our discussion of Live and Reason, these metaphors will be referred to where appropriate as Hn:

1. *When choosing metaphors based on real world objects consider the real world usability issues for insight into interface usability.*
2. *When using orientational metaphors, make sure they fit into the overall framework defined by that orientation.*
3. *When using new metaphors, carefully indicate the UI metaphorical entailments to the user.*
4. *When using conventional metaphors, be careful to indicate any deviation from the accepted set of metaphorical entailments.*
5. *Aim to use as many metaphorical entailments as possible while still keeping a firm focus on actual functionality.*
6. *Represent only those affordances of the vehicle which have an implemented function in the user-interface.*
7. *As far as possible make sure that metaphors are coherent with one another.*
8. *Ensure metaphors do not inhibit access to any functionality.*
9. *Always address the aspects of the system not explained by metaphors.*
10. *Harness the user's experience to help identifying useful vehicles.*
11. *Allow expert users to circumvent the metaphorical means of achieving tasks where such circumvention would increase task efficiency.*

3 Reason

Reason [14], aims to bring physical electronic music hardware into the realm of software. Reason emulates hardware synthesisers, mixers, samplers, drum-machines, loop players, and various effects units on general purpose computers. In Reason, each of these components are presented to the user as graphical versions of their real world equivalents, with the knobs, buttons, LEDs, cables and switches of physical hardware all reproduced.

In the physical world, electronic music hardware such as synthesisers, samplers, equalisers, effects units, and various other components are designed to be mounted vertically in standard sized racks. As a general rule, the controls for these components are situated on the face of the rack, while the sockets for wiring them together are on the rear. Reason reproduces this, with its interface dominated by a rack. Each of Reason's components are stacked vertically in this rack. Pressing the *TAB* key flips the rack around, exposing the cabling connecting the various components together. Reason's rack interface is shown on the left of figure 1, and the rear cable connection interface is shown on the right.



Fig. 1. Reason's user interface.

Metaphor: REASON IS A RACK

- *Graphical* The left and right hand sides of the window have a graphical depiction of the metal framework of a hardware rack. Devices in the rack have screws or rivets attaching them to the rack.
- *Language* The Options menu has a “Toggle Rack Front/Rear” item.

The entire design of Reason is based on the idea that it is “a virtual studio rack” [14]. While the idea of rack-based electronic music components is something that many users

will be familiar with (H10), it seems that there could be unresolved issues regarding the real world usability of racks (H1). Constraining users to a rack may create undue complexity for the user as all components must be set-up vertically. This makes it difficult for users to layout components in ways that make sense to them, and that reduce the overlaying of cables. Other products such as the Nord Modular [15] and Reaktor [16] do allow arbitrary layout of components in a two dimensional space [1] (H17). While hardware racks make sense in the physical world for physical reasons, it does not follow that a metaphor of a rack will be the best solution in the software domain.

Metaphor: A COMPONENT IS A DEVICE

- *Graphical* The appearance of each of the components are modelled very closely on physical hardware. Knobs, switches, power plugs, fuses, serial numbers and manufacturing dates are all depicted.
- *Language* The back of Reason's components feature text such as:

WARNING: To reduce the risk of fire or electrical shock, do not expose this equipment to rain or moisture...

Another curious use of language contributing to the device metaphor is that each device has been given its own branding. The drum-machine is called "Redrum", the loop player is called "Dr:rex", and the distortion unit is called "Scream 4". The fact that Propellerhead Software have created their own imaginary world of brand name hardware shows the strength of this metaphor.

Using the metaphor of a physical music device is consistent with the rack metaphor (H7), and utilises many users existing experience with music hardware (H10) through Reason's very literal interpretation. However, it seems that the real world usability issues of electronic music hardware may not have been taken fully into consideration (H1). For example, the "Redrum Drum Computer" device can only show sixteen time steps of a single drum part at a time, because it only has room for sixteen buttons. Even a simple four bar drum pattern will typically take up thirty-two time steps. The same problem is evident in the following example, where a reviewer becomes irritated with Reason's use of awkward switches in the interface [17]:

One thing that annoyed me slightly is the awkward bypass switch for the RV70000... Modelling it on a real-world switch that's awkward enough to adjust even when it's a real one surely isn't what the virtual world's supposed to be like!

Additionally, Reason's devices have depictions of superfluous switches and cables that cannot be manipulated in the interface (H6). It is possible that this could lead to some confusion for new users. Using hardware interface interaction models, rather than those standardly employed in graphical user interfaces leads to problems illustrated by the following [17]:

The eight virtual pots around the display are used to adjust those parameters, so unfortunately no easy life dragging graphics around the display.

Metaphor: THE CONNECTION BETWEEN DEVICES IS A CABLE

- *Graphical* The devices have jack cable style input and output sockets. The cables hang between these sockets and shake when they are moved.
- *Language* The Options menu has a “Show Cables” item.

The use of cables makes intuitive sense for a component connection metaphor, especially considering the extensive use of the hardware metaphor in the devices they are connecting (H7). One clear problem that emerges from the cable metaphor comes back to the use of the rack metaphor. The constraints that the rack puts on layout, coupled with the opaque appearance and non-intelligent hanging of cables is the source of very real problems (H1). Comments such as the following begin to illustrate the disadvantages of the metaphor:

The rack flips round to display the kind of spaghetti hell you would usually associate with a hardware set-up — [17]

Another consequence of the cable metaphor is that it conflicts with other potential systems of device organisation that might be more effective (H11):

The sequencer can trigger pattern changes in pattern-based modules, but you cannot switch sounds using program changes, and the sound modules aren’t multitimbral. To work around those limitations, you must run multiple instances of the modules. Doing that doesn’t appear to affect processing much, but it can make for potentially unwieldy rack configurations. *I would prefer a true matrix-switching setup, but that would probably interfere with the cabling metaphor.*” — [18] Emphasis added.

4 Live

Live by Ableton [19] is designed to “blur the separation between recording, composition and performance” [19]. It is principally a loop and sample playing tool, providing beat matching and real time audio warping, while also allowing audio input from Rewire [20] compatible audio software and the system’s audio hardware. Its interface has two main views, the *session* view and the *arranger* view.

Session The session view, shown to the left in figure 2, takes the general form of a mixing desk. It is broken into a series of vertical tracks, each of which have their own volume faders, and other audio controls. Running horizontally along the top of the mixer are a number of rows called *scenes*. A scene consists of a number of slots for audio clips, each of which corresponds to a track. Triggering a scene causes all of the scene’s audio clips to begin playing through the mixer. The user can drag and drop additional audio clips from a directory browser into scenes at any time.

Arranger The arranger view, shown on the right of figure 2, consists of a time-based view of the tracks. Unlike the session view, in this view tracks are long shallow strips laid out vertically: time now occupies the horizontal axis. The user can drag audio clips into the arranger, record audio from a sound source, or capture the output from



Fig. 2. Live's interface consisting of the Session view (left) and Arranger view (right).

interacting with Live's session view. The user can also automate a track's effects and other parameters in the arranger by reshaping graphs which are overlaid on the track.

Effects In either the session or arranger view, when the user clicks on a track, the effects associated with that track are displayed. This can be seen in the lower area of the arranger in figure 2. Effects take the output of a track, and perform various parameterised audio transformations in real time. To add an effect to a track, the user can drag and drop it from the effects library folder on the left into the track's effects area.

Audio clip When the user selects an audio clip in either the session or arranger views, an interface for manipulating the clip is displayed. This can be seen in the lower area of the session view in figure 2. The user can then modify the audio clip's various parameters. This includes the ability to *warp* the audio clip to change its inherent timing. The warping engine stretches and squeezes the audio without altering its pitch.

Metaphor: THE SESSION VIEW IS A MIXER

- *Graphical* The display has the important components and layout of an audio mixer. This includes a number of audio channels with volume faders, send controls, and a master channel.
- *Language* The Info View describes the area as the “Session Mixer”

The mixer is a very conventional metaphor in computer music tools. As Live's entire session view is based around the mixer metaphor, this takes advantage of many users' prior knowledge of how mixing desks work (H10). For example, each track on the mixer can be sent to one or more *send tracks* via the “Sends” volume controls. In Live, there is an absence of visual cues pointing to the fact that the send tracks are conceptually different from the regular tracks. Also, there is no visual indication of the signal flow from regular tracks to send tracks, other than a corresponding number or the first letter of the send track name sitting beside each send control. This is where prior knowledge of the metaphor becomes important, as users familiar with mixers will be able to rapidly identify and utilise this functionality (H10). One key place where Live does not follow an entailment of the metaphor can be seen again with the audio send functionality. Live only displays as many send volume controls as there are actual send tracks. The user can interactively add new send tracks, and each regular track will gain a new send volume

control dynamically. This can be compared to Reason's mixer, which closely follows the real world.

Metaphor: THE ARRANGER IS A MULTITRACK RECORDER

- *Graphical* The main display of the Arranger is a series of horizontal *tracks*. These show both the audio content, and controls for each track. Each track can be independently manipulated and controlled.
- *Language* The term “tracks” is used throughout the interface.

This metaphor is used very commonly in computer music recording and writing software. The multitrack recorder metaphor allows multiple tracks of audio to be recorded into the system, and then played back simultaneously. Track parameters such as volume and panning can be controlled independently of other tracks, affecting the playback of the given track. Typically, multitrack systems allow the user to *automate* track parameters over time. Live's multitracking system allows all of this.

Live's use of the multitrack recording metaphor is interesting in that it is in tension with Live's direct support of loop and sample style composition (H7). The multitrack recorder metaphor is ideal for supporting the recording and arrangement of music featuring real world instruments. However, while most sample and loop based music will be built from multiple drum tracks and bass tracks etc., the ability to throw arbitrary audio clips into the mix live does not mean that these clips should belong to a particular track.

Metaphor: AN EFFECT IS A DEVICE

- *Graphical* The effects have abstract representations of typical device controls, such as knobs, switches and faders.
- *Language* The interface refers to them as devices. They also have what Live refers to as “Factory Settings”.

The use of the effects device metaphor will suggest to users familiar with effects devices that they can be “plugged in” to other components, and that the effects will have settings that can be manipulated (H10). However, the metaphor is kept at a quite abstract level. Instead of wiring an effect up between an arbitrary sound source and destination, Live simply provides a mechanism to associate an instance of an effect device with a particular track. This is analogous to *effect inserts* in real world mixers. This means that you are limited to as many effects processing flows as you have tracks. This issue is discussed further in section 5.

Metaphor: SOUND IS A WAVE FORM

- *Graphical* When a user selects an audio clip, a waveform representation of the audio is shown in the sample display area. This can be seen in the lower part of the session view in figure 2. The arranger view also shows waveforms as representations of the audio clips, placing a graphic of the waveforms in the time and track where the audio is located.

The waveform is a conventional metaphor for computer tools that deal with audio. Originating from oscilloscope displays, this metaphor is based on its component orientational metaphors *AMPLITUDE IS WIDTH* and *THE PASSAGE OF TIME IS RIGHT*. As this is such a common metaphor, the majority of non-novice users should be able to utilise their understanding of waveforms in Live's interface (H10). This metaphor is of central importance in Live as it serves as the principle interface for flexible real-time audio warping. When a user selects an audio clip, a detailed display of the audio is shown as a waveform. The user can use *warp markers* to tell Live where they want the beats of the bar to be in relation to the shape of the waveform. For example, if the user sees a large jump in the width of the waveform, they may deduce that this is the kick-drum they can hear when they play the sample. If they want the kick-drum to land on the third beat of the bar, they can drag the warp marker labelled 3 onto the large jump. However, this begins to break down the orientational metaphor of *THE PASSAGE OF TIME IS TO THE RIGHT*. Because time, as the user perceives it, is now indicated by the location of the warp markers, rather than by horizontal position, the usage of the waveform metaphor is deviating from its accepted set of behaviours (H4). While this interface is comprehensible, we would expect an alternative interface that preserved the accepted behaviour of the waveform metaphor might be more appropriate and intuitive.

5 Comparison

Live and Reason solve similar problems with a mix of comparable and differing techniques. The following sections compare these techniques with a focus on how metaphors are utilised.

Shared conventional metaphors Reason and Live share a large number of conventional metaphors. Both utilise the knob metaphor, where the user clicks on a knob and drags up or down, which causes the knob to rotate. A convention has developed in music interfaces where *UP IS CLOCKWISE* and *DOWN IS ANTI-CLOCKWISE*, in line with corresponding metaphors for the parameters that the knob is controlling. Both applications also feature the mixer metaphor as an important part of their organisation, including the standard fader volume control which can be moved vertically to modify volume. These interfaces include the knob, fader and mixer metaphors, as well as the button metaphor. There are a large number of other minor metaphors that Live and Reason have in common, but it is not practical to enumerate them all here.

Music hardware metaphors Both Reason and Live utilise electronic music hardware metaphors. While Live's interface uses these metaphors to a lesser extent, the example of Live's mixer send tracks, discussed in section 4, shows that it is also dependent on leveraging peoples real world knowledge. What may become a problem is that as music software becomes increasingly powerful, it is likely that it will begin to replace music hardware altogether. An important question that must be answered to validate the dependence on music hardware metaphors is: what proportion of new and potential users have prior experience with music hardware?

Realism Vs. Abstraction Both Reason and Live incorporate hardware metaphors into their interfaces. However, the obvious difference between them is how tightly they mimic their real world counterparts. Reason closely follows its hardware counterparts,

while Live takes a more abstract approach, and this is apparent in the visual cues of their interfaces. The attention to visual accuracy in Reason's interface is a strong cue that the behaviour of the interface will match physical hardware devices.

One example of Live and Reason's different approaches can be seen in how Live associates effects devices with a particular track. This utilises a common approach used in user interfaces where one element is shown as a property of parent element. In Live, when you select a track, the effect properties of that track are shown. Contrasted with Reason, where all effects are on the screen at all times, linked by a complicated network of cabling, it is clear that Live has a more elegant approach that is conventional in software interfaces.

The principle problem with Reason's literal approach is that you keep all of the disadvantages of physical hardware, without utilising the advantages that software interfaces offer. The example of Live's dynamic mixer, compared to Reason's static "physical" mixer is an example of how Live exploits the flexible nature of software while Reason sticks to resemblance. The latter may be less useful, but it can still generate enthusiasm [17]:

The cables even jiggled as you moved them. This in itself was a nice touch, but more importantly illustrated the kind of over-engineering and attention to detail we've come to expect from this Swedish software company.

6 Conclusions

The new applications *Reason* and *Live* show that metaphor is still an important part of real world interface design. In this paper, we have used our semiotic theory of metaphors, and the heuristics derived from that theory, to compare and contrast *Reason* and *Live*, two integrated electronic music production systems. We chose electronic music production for this study as it is a relatively new application area, and so the metaphors found in its user interfaces have not yet succumbed to conventionality. We demonstrated that our analyses were able to uncover significant similarities and differences between the two tools and the way they utilised metaphor. More importantly, we were also able to provide principled rationales to explain these similarities and differences. We expect these analyses could assist musicians trying to make sense of some of the interface design decisions found in Reason and Live; more pragmatically, we believe they will be applicable to the designers of next-generation electronic music production systems, as well as to any interface designer who intends to utilise metaphor.

References

1. James Noble and Robert Biddle. *Software Visualization*, K. Zhang(Ed.), chapter Visual Program Visualisation. Kluwer, 2003.
2. James Noble and Robert Biddle. [Visualising 1,051 visual programs: Module choice and layout in nord modular patch language.](#) In John Grundy and Paul Calder, editors, *User Interfaces 2002*. Australian Computer Society, 2002.

3. James Noble and Robert Biddle. Program visualisation for visual programs. In Peter Eades and Tim Pattison, editors, *Information Visualisation 2001*. Australian Computer Society, 2001.
4. Jeff Johnson, Teresa L. Roberts, William Verplank, David C. Smith, Charles H. Irby, Marian Beard, and Keven Mackey. The Xerox Star: A retrospective. *IEEE Computer*, 22(9):11–29, September 1989.
5. Steven Johnson. *Interface Culture: How New Technology Transforms the Way We Create and Communicate*. Harper SanFrancisco, 1997.
6. Apple Computer, Inc. Staff. *Macintosh Human Interface Guidelines*. Addison-Wesley, 1992.
7. Microsoft Corporation. *The Windows Interface Guidelines for Software Design: An Application Design Guide*. Microsoft Press, 1995.
8. John M. Carroll and Robert L. Mack. Learning to use a word processor: By doing, by thinking, and by knowing. In Ronald M. Baecker, Jonathan Grudin, William A. S. Buxtin, and Saul Greenberg, editors, *Readings in Human-Computer Interaction: Toward the Year 2000*, pages 698–717. Morgan Kaufmann Publishers, Inc., 1995.
9. Donald A. Norman. *The Invisible Computer*. The MIT Press, 1998.
10. Pippin Barr, James Noble, and Robert Biddle. *Virtual, Distributed and Flexible Organisations*, chapter A Semiotic Model of User-Interface Metaphor. Kluwer Academic Publishers, 2003.
11. Pippin Barr, Robert Biddle, and James Noble. A taxonomy of user interface metaphors. In *Proceedings of SIGCHI-NZ Symposium On Computer-Human Interaction (CHINZ 2002)*, Hamilton, New Zealand, 2002.
12. Pippin Barr. User-interface metaphors in theory and practise. Master’s thesis, Victoria University of Wellington, 2003.
13. George Lakoff and Mark Johnson. *Metaphors We Live By*. The University of Chicago Press, 1980.
14. Propellerhead Software. Reason, 2003. <http://www.propellerhead.se/products/reason/>.
15. Clavia Digital Musical Instruments. Nord modular & nord micro modular, 2003. <http://www.clavia.se/nordmodular/index.htm>.
16. Native Instruments. Reaktor 4 - the future of sound, 2003. http://www.native-instruments.com/index.php?reaktor4_us.
17. Future Music. Propellerhead reason 2.5. *Future Music*, FM136:30–37, June 2003.
18. Jeff Burger. Propellerhead reason 1.0 (mac/win). *Electronic Musician*, 17 i7:164, July 2001.
19. Ableton. Live: The audio sequencing instrument, 2003. <http://www.ableton.com/index.php?main=live>.
20. Propellerhead Software. Rewire, 2003. <http://www.propellerhead.se/products/rewire/>.